Building scalable apps Lesson

1. Introduction

Building applications that can **scale effortlessly** requires continuous **improvement, innovation, and best practices**. This lesson takes you through the journey of **developing a single-user application** and then scaling it to serve **millions of users** efficiently.

2. Learning Objectives

By the end of this lesson, you will:

- Explain how **web applications** function.
- Set up a **single-server architecture** for web applications.
- Understand the **role and importance of load balancers**.
- Describe **database replication** and its impact on performance.
- Implement **caching strategies** to enhance performance.
- Explain how **Content Delivery Networks (CDNs)** optimize content delivery.
- Differentiate between **stateful and stateless applications**, and why stateless architecture is beneficial for scaling.
- Apply **scaling strategies** to support high-traffic applications.

3. Scaling Web Applications

3.1 Understanding Web Applications

A typical web application consists of:

- **Frontend**: The user interface (HTML, CSS, JavaScript)
- **Backend**: The logic and data processing (Node.js, Python, Java, etc.)
- **Database**: Storage of application data (SQL, NoSQL databases)

3.2 Single-Server Setup

Ask AI Tutor

A simple application can run on a **single server**, but this setup has **scalability limitations**. Common bottlenecks include:

- CPU and memory constraints.
- Database overload.
- Increased response time as users grow.

3.3 Load Balancers

A **load balancer** distributes traffic across multiple servers to:

- Prevent a single point of failure.
- Optimize resource utilization.
- Ensure **high availability** and **fault tolerance**.

3.4 Database Replication

Database replication helps scale data storage by:

- Creating **multiple copies** of a database.
- Allowing read requests to be **distributed across replicas**.
- Enhancing availability and disaster recovery.

3.5 Caching for Performance

Caching reduces server load and speeds up responses by storing frequently accessed data:

- **Application Caching** (e.g., Redis, Memcached).
- **Database Caching** (e.g., query caching in MySQL, PostgreSQL).
- **CDN Caching** for static content.

3.6 Content Delivery Networks (CDNs)

A **CDN** helps serve content from distributed edge locations **closer to the users**, reducing latency and improving performance.

3.7 Stateful vs. Stateless Applications

- **Stateful Applications** require user session data to be stored on a server, making scaling more complex.
- **Stateless Applications** store session data externally (e.g., in databases or caches), allowing easy scaling across multiple servers.

3.8 Scaling Strategies

1. **Vertical Scaling (Scaling Up)**: Adding more CPU, RAM, or storage to a single server.
2. **Horizontal Scaling (Scaling Out)**: Adding more servers to distribute the load.

3. **Microservices Architecture**: Breaking the application into smaller, independent services.
4. **Database Sharding**: Splitting large datasets across multiple databases to balance the load.

4. Additional Resources

For a deeper dive into scaling strategies, check out **Scaling From Zero to Millions of Users**.

🚀 **Congratulations!** You now have the foundational knowledge to build and scale applications efficiently!

AI TUTOR ACTIONS

Generate a practice quiz Summarize this unit Ask about this unit

Complete unit

Previous

Week 2: Day 1
Introduction to Cloud Computing

Next